EE3111: Statistics for Electrical Engineers

IPL Powerplay Prediction Project (Optional)

Jan–May 2026

Your task is to build a prediction model for the IPL T20 (men) competition:

> **Predict the total runs scored by the batting team at the end of 6.0 overs (Powerplay), using information available up to the end of 3.0 overs.**

Inputs (X): Ball-by-ball sequence of first three overs *plus* match information such as teams, venue, toss outcome, etc for a given match. You are also allowed access to any data about previous matches available on the internet to incorporate in your model.
Output (Y): Runs at 6.0 overs in the **first innings**.

Submission Rounds and Live Evaluation Window

You will submit your model **three times**. Each submission is "frozen" and evaluated on the **first innings** of IPL matches during the specified window.

Round	Submission deadline (IST)	Used for live evaluation on matches
Round 0 (Trial)	Apr 13–17	Apr 13–17 (first innings only)
Round 1	Apr 20, 6:00 PM	Apr 20–26 (first innings only)
Round 2	Apr 27, 6:00 PM	Apr 27–May 10 (first innings only)

Important: Round 0 is a **trial run**. Your leaderboard-based project marks are computed using **Rounds 1 and 2 only**.

Model Interface

Please follow the below instructions to ensure that you are evaluated correctly. An automatic script is used to evaluate your leaderboard scores.

1. You must submit a single `zip` folder that is named after the roll numbers of the members in your group separated by an underscore. Example: `EP21B015_EP21B016_EP21B017.zip`
2. Inside the zip folder, create a `main.py` file. You can possibly have other files in the folder.
3. Inside the `main.py` file, define a `predict(match_data, ball_by_ball_data)` function. The function should return a 'float' object.
4. Now, `match_data` and `ball_by_ball_data` are both strings in the Ashwin CSV format, but with two important changes:

- The `match_data` is the string form of a match info file in the `Ashwin` CSV format. However, the information pertaining to the winners of the match (i.e. the fields 'info,winner', 'info,winner_runs,', etc.) have been removed. You **cannot** use them in your prediction function.
- The `ball_by_ball_data` is the string form of a ball-by-ball file in the `Ashwin` CSV format. However, it contains the data of only the first three overs in the first innings, including extras. Only these balls are included and attempting to access any more overs will lead to errors.

The data in the above two files is passed to the functions as a Python `string` object, not as a file itself. Thus, the `predict` function takes as arguments two strings and returns a float.

5. You can use the data from previous matches available on the internet to make predictions. You are **not allowed** to train on the data during evaluation, since it might take too much time. You can save your statistical / ML model as say a pickle file or a numpy array.

   Please **do not** include all the training data in your submission, only your final model is required.

6. Ensure that your submission runs within a few seconds and takes up only a few MB of space for each match.

You will have to upload the file using a Google form, which will be shared later.

Example

Consider the below code (`main.py` file) which takes the score at the end of the first three overs and simply doubles it as the output.

```
from io import StringIO
import pandas as pd

def predict(match_data, ball_by_ball_data):
    # Convert the CSV string into a pandas DataFrame
    df = pd.read_csv(StringIO(ball_by_ball_data))

    # Convert runs and extras to numeric, replace NaNs with 0
    df["runs_off_bat"] = pd.to_numeric(df["runs_off_bat"], errors="coerce").fillna(0)
    df["extras"] = pd.to_numeric(df["extras"], errors="coerce").fillna(0)

    # Compute total score after 3 overs (runs + extras)
    score_after_3_overs = int((df["runs_off_bat"] + df["extras"]).sum())

    # Predict final score using a simple doubling heuristic
    return 2.0 * score_after_3_overs
```

Evaluation Metric: RMSE

For each evaluation window, your predictions are compared to the true 6-over score on each match (first innings). Error is measured using RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2},$$

where y_i is the actual score and $\hat{y}_i$ is the predicted score for the ith match. Lower RMSE is better.

Marks and Grading Policy

Weightage

The project is optional and has **20 marks**. The final exam has **60 marks** (6 questions of 10 marks each).

How the project affects your final exam score

Let the six exam question scores be $q_1, \ldots, q_6$, each out of 10 with $q_{(1)}, \ldots, q_{(6)}$ being their decreasing order version. Let P be your project score out of 20.

Your **final marks out of 60** are computed as:

$$q_{(1)} + q_{(2)} + q_{(3)} + q_{(4)} + \max(P, q_{(5)} + q_{(6)})$$

Interpretation: your two weakest exam questions (total L) can be replaced by $\max(P, L)$. This can only improve (or leave unchanged) your score, never reduce it.

Project marks breakdown (out of 20)

- **10 marks: Leaderboard position** (computed using **Rounds 1 and 2** performance only; lower RMSE ranks higher).
- **10 marks: Statistical ideas used** to devise the model (feature engineering, validation strategy, regularization, uncertainty, baselines, etc.).